PRODUCT & ENGINEERING BLUEPRINT

AI Workflow SaaS Implementation Guide

From a customized WhatsApp MVP to a broader business automation platform.

Prepared for Dursun Kaplıca

Version 1.0 | 14 September 2026

31 implementation sections | 8 phases | Iterative delivery

BUILD THE REPEATABLE CORE FIRST

Guided setup. Tested workflows. Connected tools. Visible results. Controlled expansion.

Planning assumptions are marked. Provider references are linked. Every phase includes a completion gate.

Guide map

Read sections 01-07 for the product, 08-18 for the technical design, and 19-29 for delivery and growth.

01 Product definition and assumptions	3
02 Scope: MVP through broader vision	4
03 Roadmap and iteration sequence	5
04 Discovery and market validation	6
05 People, screens, and user journeys	7
06 Questionnaire and configuration model	8
07 Three MVP workflows	9
08 Application architecture and stack	10
09 Database design and relationships	11
10 API and event contracts	12
11 Workflow engine and state machine	13
12 Reliability and recovery	14
13 WhatsApp integration	15
14 Calendar, Sheets, email, and forms	16
15 AI behavior and evaluation	17
16 Security, privacy, and tenant isolation	18
17 Billing, pricing, and operating economics	19
18 Environments, deployment, and operations	20
19 Foundation build tasks	21
20 MVP workflow build tasks	22
21 Paid pilots and product validation	23
22 Self-service product and second template	24
23 Configurable workflow platform	25
24 Broader vision and long-term expansion	26
25 Test plan and release acceptance	27
26 Launch and marketing playbook	28
27 Metrics, experiments, and decision rules	29
28 Support, customer success, and risk register	30
29 First 30 days and delivery checklist	31
30 Glossary and decision log	32
31 Official references and implementation notes	33

01 / START HERE

Product definition and assumptions

Product promise

- A business answers guided questions, connects its accounts, reviews a proposed workflow, tests it, and activates it. The product then performs agreed tasks and shows what happened.
- The first product handles WhatsApp customer inquiries, lead capture, appointments, and staff handoff. The broader product supports configurable workflows across forms, email, spreadsheets, calendars, CRMs, and other approved tools.
- Business owners buy fewer missed inquiries and less administration. The product must show these outcomes, not merely count AI messages.

Working assumptions - confirm during discovery

- Initial customer: small cleaning businesses with one location, one shared booking calendar, and repeatable services. This is a planning choice, not a user-confirmed niche or proven market opportunity.
- Builder: one full-time developer comfortable with web development. Founder also owns interviews, onboarding, support, and sales. Use outside design, security, and legal expertise at specific review points.
- First interface: responsive web dashboard. Customer channel: WhatsApp text. First pilot language: choose the language used by the first customers; validate Arabic separately before advertising support.
- Initial scheduling: fixed service durations, one calendar resource, business-defined buffers. Multi-crew dispatch and travel-time routing come later.
- All timelines, prices, targets, budgets, and thresholds in this guide are proposed planning assumptions, not revenue forecasts or quoted vendor prices.

How to use this document

- Read the roadmap first. Implement iterations in order and pass the exit gate before expanding scope.
- Turn each implementation bullet into a tracked task with an owner, acceptance check, and evidence link. Unresolved product choices belong in a decision log.
- This is an implementation blueprint, not a completed codebase. Confirm current API rules, provider access, and prices before committing to production contracts. Research checked 14 September 2026.

02 / PRODUCT BOUNDARIES

Scope: MVP through broader vision

Paid MVP - phases 0 to 3

- Guided setup; business profile; three fixed workflow templates with editable fields and rules; one WhatsApp number per business.
- Lead capture to the internal database and Google Sheets; booking through Google Calendar; staff notification through a transactional email provider.
- Conversation inbox, manual takeover, approval queue, test mode, workflow version history, basic billing, usage limits, and recovery tools.
- Allow public intake forms to collect missing information. Initially these forms are generated by your product, not arbitrary third-party websites.

Expansion - phases 4 and 5

- More tested industry templates; conditional branches; scheduled follow-ups; more calendars and team resources; customer-facing forms with attachments.
- Inbound and outbound business email, CRM connections, external form triggers, web chat, reusable actions, and controlled custom webhooks.
- AI turns a business description into a draft workflow assembled from supported actions. The owner reviews and tests every published change.

Broader platform - phases 6 and 7

- Multi-location operations; partner-managed client accounts; curated template marketplace; advanced roles and approvals; deeper reporting.
- Customer service, onboarding, document collection, lead qualification, staff tasks, invoice-request follow-ups, and other validated business processes.
- Optional voice, image extraction, and document processing after measuring demand, accuracy, and unit cost. Native mobile apps only when dashboard usage proves a need.

Explicit limits

- Do not promise that a questionnaire can automate every business process. Unsupported actions remain visible as unavailable or require a separately scoped service.
- Do not launch arbitrary generated code, unrestricted browser agents, autonomous payments/refunds, or regulated decisions in the MVP.
- Business variation should mostly change configuration. If each customer needs custom code, price it as implementation work and study whether the pattern deserves a product feature.

03 / DELIVERY PLAN

Roadmap and iteration sequence

Sequence and estimated effort

- Phase 0 - discovery and access checks: iteration 0, 1-2 weeks. Confirm buyer pain and provider onboarding feasibility.
- Phase 1 - foundation: iterations 1-2, 3-4 weeks. Build tenant accounts, setup flow, test harness, and connector foundations.
- Phase 2 - working MVP: iterations 3-5, 5-7 weeks. Deliver intake, booking, handoff, runtime recovery, and billing.
- Phase 3 - paid pilots: iterations 6-7, 3-4 weeks. Operate with real customers, fix failures, and test willingness to renew.
- Phase 4 - self-service product: iterations 8-9, 4-6 weeks after the pilot gate. Reduce onboarding support and add a second template.
- Phase 5 - configurable platform: iterations 10-12, 6-10 weeks after repeatable demand. Add bounded workflow editing, more connectors, and AI-assisted configuration.
- Phase 6 - industry and partner expansion: iterations 13-14, 6-10 weeks if retention and margins justify it.
- Phase 7 - enterprise and broader automation: iteration 15 onward. Scope from signed demand; do not set a fixed solo-build completion date.

Capacity and dependencies

- Plan roughly 12-17 full-time weeks through pilots, plus provider approval delays. A beginner or part-time schedule will take longer.
- Reserve 20-30% of each iteration for customer conversations, support, fixes, and integration surprises. Do not spend all capacity on new features.
- Start WhatsApp production onboarding, Google OAuth verification assessment, and payment-provider eligibility checks in phase 0; external approval is not controlled by coding speed.

Every iteration follows the same loop

- Pick one customer outcome; write acceptance examples; build the smallest end-to-end slice; test failures; demo it; record feedback; decide whether to continue.
- Use a staging environment first. Pilot releases go to one tenant before broader rollout.
- At each phase gate, choose continue, narrow, or pause. Failing a gate means fix the relevant problem rather than adding more integrations.

04 / PHASE 0 - ITERATION 0

Discovery and market validation

Interview and workflow research

- Create a list of 30 suitable cleaning businesses using public listings, referrals, and your network. Prioritize businesses you can speak with directly.
- Interview 10 owners or operations staff. Ask them to walk through their last inquiry, booking, exception, and missed follow-up.
- Record weekly inquiry volume, staff minutes per inquiry, current tools, typical errors, buying authority, budget, and why previous automation failed.
- Ask for permission to inspect anonymized examples. Map the actual procedure and exceptions; do not use customer data in a demo without authorization.
- Compare at least five alternatives: direct WhatsApp booking tools, general automation platforms, human assistants, existing booking software, and doing nothing. Note setup effort and customer switching costs.

Validate the offer before broad development

- Demonstrate one sample inquiry that collects details, writes a record, proposes a slot, and notifies staff. Use a clearly labeled simulated demo.
- Offer a limited paid pilot with defined workflows, supported tools, usage cap, setup assistance, support hours, and cancellation terms.
- Ask for a payment or signed pilot commitment. Compliments and email signups alone do not demonstrate willingness to pay.
- Create a simple landing page with one industry, one outcome, a short demo, and a book-a-demo form. Track qualified demos and pilot conversions.

Access checks and exit gate

- Verify how you may onboard third-party WhatsApp businesses through your chosen provider. Test a sandbox message and a production onboarding path. [S1]
- Confirm that a Google account can authorize the calendar and sheet scopes you need. Document verification requirements before public launch. [S2, S3]
- Confirm payment processing is available for your actual legal entity and bank account; do not assume a particular provider supports your country.
- Exit gate: three credible pilot commitments, one repeated process shared by those businesses, and a feasible connector path. If this fails, change the niche or offer before building phase 1.

05 / PRODUCT EXPERIENCE

People, screens, and user journeys

Roles

- Owner: manages subscription, accounts, team access, data export, and workflow publishing.
- Manager: updates business settings, reviews approvals, and operates conversations. Publishing permission can be granted explicitly.
- Operator: replies to customers and resolves assigned exceptions. Cannot view connector secrets or change billing.
- Platform support: sees technical health by default. Any access to customer content must be restricted, time-limited, and recorded.

Required screens

- Public website: industry use case, demonstration, pricing assumptions replaced by live plans, FAQs, privacy notice, terms, and contact.
- Account area: sign-up, sign-in, password reset, email verification, organization creation, invitations, and onboarding progress.
- Setup: business details, selected tasks, service rules, knowledge, integrations, workflow preview, test, and activation.
- Daily workspace: overview, conversations, leads/bookings, approvals, workflow runs, integrations, settings, billing, and usage.
- Operator inbox: conversation, extracted details, active workflow, pending action, assignment, pause/resume, and send-message control.
- Error views: disconnected account, expired authorization, rejected message, unsupported request, payment issue, and unavailable service. Each offers a next step.

Experience rules

- Show outcome language: customer saved, booking pending, staff action needed. Put technical details behind a troubleshooting panel.
- Save onboarding after every step. Allow back navigation without losing answers; show required versus optional inputs.
- Display dates in the business timezone and show timezone explicitly when customers may be elsewhere.
- Design keyboard access, labeled inputs, visible focus, readable contrast, and mobile layouts. Keep messages understandable without relying on color.
- Show a setup preview before charging for activation. If setup is blocked, explain why and avoid silently starting a paid billing period.

06 / GUIDED CUSTOMIZATION

Questionnaire and configuration model

Questions and stored fields

- Identity: business_name, industry, country, currency, timezone, language, public_contact, and legal business details needed by providers.
- Services: service_id, description, fixed_price_minor, duration_minutes, buffer_minutes, allowed_areas, and active status.
- Operations: working hours, holidays, minimum booking notice, maximum booking horizon, required customer fields, cancellation policy, and staff contact.
- Tasks: enable lead capture, booking, staff review, and supported follow-up templates. Ask where each result should go.
- Knowledge: approved FAQs, service exclusions, business links, reply tone, escalation topics, and approved example answers.
- Permissions: which actions can run automatically, which need approval, who approves, and how long a pending request remains valid.

Validation and compilation

- Use structured form fields for prices, schedules, and rules. Free-text answers can suggest settings but must not silently override validated values.
- Store answers as a versioned BusinessConfig JSON object. Validate with a shared schema on both server and client; server validation is authoritative.
- Reject invalid timezone names, overlapping working-hour ranges, negative prices, missing destinations, and unsupported action combinations.
- Compile approved configuration into a selected workflow template. Record template version, config version, connector bindings, and unresolved questions.
- Show a preview: trigger, required information, conditions, actions, approval points, and destinations. Highlight differences from the live version.

Completion checks

- A business can finish setup without writing a prompt or understanding APIs. Each question includes a short example.
- Unknown or contradictory answers produce a clarification question; never invent service prices or business policies.
- Store progress as draft until connections and test cases pass. Re-authentication must not erase the draft.
- Changing business rules creates a new draft. Running workflows keep their original version unless explicitly cancelled.

07 / FUNCTIONAL BEHAVIOR

Three MVP workflows

W1 - inquiry to lead record

- Trigger: a new customer WhatsApp message. Identify the request and ask for missing name, service, area, and preferred contact details.
- Validate service and service area against business rules. Unsupported requests go to W3.
- Create or update a tenant-scoped lead in Postgres; mirror it to the configured Google Sheet; notify staff by email.
- Confirm receipt only after the internal lead is saved. A failed sheet update remains visible and retries without losing the lead.
- Acceptance: repeated delivery of the same message produces one lead event and no duplicate staff notification.

W2 - inquiry to confirmed appointment

- Collect W1 fields plus preferred date/time. Resolve an exact timestamp and ask for clarification when a date is ambiguous.
- Query availability, apply working hours and duration rules, and offer a small set of slots. Do not claim a slot is reserved unless a hold exists.
- After explicit customer confirmation, recheck the slot and create the calendar event. Save the event ID, booking ID, and booking status.
- Send confirmation only after the booking is confirmed. Mirror the result to Sheets and notify staff as separate recoverable steps.
- Acceptance: two customers requesting the same internal resource cannot both receive a confirmed booking from your system.

W3 - exception to human review

- Trigger: complaint, unusual pricing, unsupported service, unclear request after two clarification attempts, failed dependency, or customer asks for a person.
- Pause automated replies for that conversation; save a summary with original message links; assign staff and send a notification.
- Staff can reply, approve a proposed action, reject it, or request more information. Resume automation only through a deliberate control.
- Acceptance: an operator taking over stops queued AI replies from being sent; an expired approval cannot execute a stale booking.

08 / TECHNICAL FOUNDATION

Application architecture and stack

Recommended starting architecture - design choices

- Use a TypeScript monorepo with a modular web application and a separate background worker. Avoid microservices until scaling or ownership requires them.
- Web: Next.js with React for the dashboard and server endpoints. Shared validation: Zod. Database migrations: SQL files tracked in Git.
- Data: managed Postgres, with Supabase Auth and private object storage as a practical managed starting option. Tenant isolation must be explicit. [S7]
- Execution: BullMQ with managed Redis for background jobs, retries, and delayed work. Postgres remains the durable record of business state. [S6]
- AI: a server-only model adapter supporting structured responses. Choose a model through evaluation; keep model names, limits, and prices configurable.
- Providers: WhatsApp through one approved provider path, Google Calendar and Sheets, one transactional email service, and an eligible subscription payment provider.

Data flow

- Customer message reaches the verified webhook endpoint. The endpoint saves an event and outbox row in one database transaction, then acknowledges receipt.
- An outbox dispatcher creates a worker job. The worker resolves the business, loads the pinned workflow, extracts information, evaluates rules, and executes allowed actions.
- Action results and provider IDs are stored before later steps proceed. The dashboard reads internal state; callbacks update delivery status.
- A scheduler finds due reminders, expired approvals, and stuck runs. All scheduled work checks current permissions and cancellation state before acting.

Repository layout and deployment

- apps/web: UI and API routes. apps/worker: queues, dispatcher, and scheduler. packages/domain: business rules and state transitions.
- packages/connectors: provider adapters. packages/workflows: schemas and templates. packages/ai: prompts, evaluation, and model adapter. db/migrations and tests hold verification assets.
- Deploy the web app on a supported Node host and the worker as a long-running container. Keep database, Redis, and compute regions aligned where feasible.
- Lock dependency versions, document the runtime version, and upgrade through staging. Avoid running long workflows inside short-lived request handlers.

09 / CORE DATA MODEL

Database design and relationships

Identity and configuration tables

- organizations(id, name, timezone, currency, status); users managed by the auth provider; memberships(org_id, user_id, role). Unique membership per organization/user.
- business_configs(id, org_id, version, config_json, created_by, created_at); services(id, org_id, config_version, duration, price_minor, active).
- connections(id, org_id, provider, external_account_id, scopes, status, secret_reference, expires_at); connector_bindings map approved calendar, sheet, and sender IDs.
- workflow_templates(id, version, schema_json); workflow_versions(id, org_id, template_id, version, definition_json, status, published_by). Published versions are immutable.

Customer and execution tables

- contacts(id, org_id, channel, external_contact_id, consent_status); conversations(id, org_id, contact_id, state, assigned_user_id, last_inbound_at).
- messages(id, org_id, conversation_id, provider_message_id, direction, content_ref, delivery_state, occurred_at). Unique provider/sender/message identity.
- leads(id, org_id, contact_id, service_id, details_json, status); bookings(id, org_id, lead_id, resource_id, starts_at, ends_at, status, external_event_id).
- webhook_events(id, provider, external_event_key, org_id, payload_ref, received_at, processed_at); outbox(id, org_id, event_id, kind, available_at, dispatched_at).
- workflow_runs(id, org_id, workflow_version_id, correlation_id, state, context_json); step_runs(id, org_id, run_id, node_id, attempt, state, error_code).
- action_receipts(id, org_id, operation_key, provider, external_id, request_hash, state); approvals(id, org_id, run_id, proposed_action_hash, expires_at, decision).

Operations tables and invariants

- usage_events, subscriptions, audit_events, knowledge_items, form_submissions, and integration_sync_jobs each carry org_id and timestamps.
- Use UUID internal keys, timestamptz dates, integer monetary minor units plus currency, and explicit state enums/check constraints.
- Index org_id with status and created_at for inboxes and run lists. Make operation_key unique per organization.
- Use composite foreign keys or equivalent checks to prevent cross-organization relationships. Encrypt secrets separately; never place refresh tokens in ordinary JSON fields.
- Add retention dates to message payloads and logs. Store files privately with tenant-scoped access, type checks, and signed short-lived links.

10 / BACKEND INTERFACES

API and event contracts

Dashboard endpoints

- POST /api/organizations and /api/invitations; GET/PATCH /api/business-config; POST /api/business-config/validate. Require authenticated membership and the appropriate role.
- POST /api/connections/{provider}/authorize; GET /api/connections/{provider}/callback; POST /api/connections/{id}/disconnect. Bind OAuth state to the initiating user and organization.
- POST /api/workflows/preview; POST /api/workflows/{id}/test; POST /api/workflows/{id}/publish; POST /api/workflows/{id}/pause.
- GET /api/conversations with cursor pagination; POST /api/conversations/{id}/takeover; POST /api/conversations/{id}/messages; POST /api/conversations/{id}/resume.
- POST /api/approvals/{id}/decide; GET /api/runs/{id}; POST /api/runs/{id}/retry-step. Retry permission is separate from ordinary read access.
- POST /api/billing/checkout; GET /api/usage; POST /api/exports; DELETE /api/organization with re-authentication and explicit confirmation.

External endpoints and events

- POST /webhooks/whatsapp, /webhooks/billing, and /webhooks/email: validate provider signature using the exact required request representation before processing.
- GET /forms/{token} and POST /forms/{token}/submit: scoped token, expiry, rate limit, input validation, and duplicate submission protection.
- Internal event envelope: event_id, event_type, schema_version, org_id, subject_id, occurred_at, correlation_id, causation_id, payload.
- Event types: message.received, lead.saved, booking.confirmed, approval.requested, action.failed, connection.revoked, and subscription.updated.

Contract rules

- Validate JSON and reject unknown action types. Derive organization access from authenticated membership; never trust a caller-supplied org_id alone.
- Use idempotency keys for externally visible write requests. Return a stable operation ID for queued work, not a false success result.
- Return a consistent error body: code, safe_message, request_id, retryable, field_errors. Suggested status codes: 400 invalid input, 403 forbidden, 409 conflict, 429 rate limit.
- Document contracts in OpenAPI and generate typed client bindings. Version webhook adapters independently of business workflow schemas.

11 / EXECUTION MODEL

Workflow engine and state machine

Workflow definition

- Represent each workflow as versioned JSON: id, schema_version, trigger, input_schema, nodes, edges, policy, and connector_bindings.
- MVP node types: collect_fields, classify, condition, save_lead, sheet_upsert, check_slots, create_booking, send_message, send_email, approval, and finish.
- Each node declares required fields, input mapping, output schema, timeout, retry policy, and whether it changes external state.
- Validate the graph before publish: supported nodes, reachable end, valid mappings, available connections, and approval requirements. MVP graphs are fixed templates.

Runtime behavior

- Run states: queued, running, waiting_input, waiting_approval, succeeded, failed, cancelled. Step states also include uncertain for an ambiguous external result.
- Acquire a short conversation lease with a fencing token; process messages in a controlled sequence. Recheck the token before any outward action.
- Load the workflow version pinned at run creation. Save validated context after every step so a worker restart resumes from a durable checkpoint.
- Waiting for a customer or approval persists state and releases the worker. A later event schedules continuation; do not keep a process asleep for hours.
- Limit run steps, elapsed lifetime, message frequency, and AI budget. Exceeding a limit creates an operator task instead of an uncontrolled loop.

Publishing and changes

- Draft -> validated -> tested -> published -> paused/archived. Only a permitted owner or manager publishes.
- Before publish, show connection destinations, allowed external actions, sample results, and changes from the current version.
- New runs use the new version. Old runs retain old business rules; cancellation and urgent safety controls override both.
- Rollback selects a previous version for new runs. It does not reverse emails already sent or calendar events already created.
- For future graph editing, begin with an acyclic graph and bounded list iteration. Add loops only with explicit limits and reliable resume semantics.

12 / AVOID DUPLICATE OR LOST WORK

Reliability and recovery

Durable processing

- Commit inbound event and outbox row atomically. If the process crashes before queue dispatch, a later dispatcher still finds the row.
- Deduplicate using provider account, event type, and stable event identity. Do not rely solely on queue job IDs because removed jobs may no longer prevent duplicates. [S6]
- Maintain an action ledger with an operation key derived from organization, run, node, and logical action version. Store provider results and the request hash.
- Use provider idempotency support where available. When a timeout leaves an outcome unknown, reconcile using provider IDs or a lookup before retrying.
- Do not claim universal exactly-once delivery: external systems differ. For ambiguous email or sheet writes without safe reconciliation, mark uncertain and request review.

Failure policy

- Transient errors: bounded exponential backoff with jitter; honor Retry-After. Initial proposal: five attempts over up to 15 minutes, configurable by action.
- Permanent errors such as revoked permission or invalid destination: stop the affected action and notify the owner with a reconnect or repair link.
- Provider outage: open a circuit breaker, delay work, and surface degraded status. Apply per-provider and per-tenant concurrency caps.
- A dead-letter queue holds exhausted jobs with redacted error details. Operator retries only the failed action after checking earlier receipts.
- Scan for stuck leases and long-running operations. Reclaim safely with fencing; never let two workers independently send the same pending action.

Business consistency

- Booking is the primary outcome; sheet mirroring and staff email are secondary. Their failure must not cause a second booking.
- Cancellation is a new explicit operation, not an automatic side effect of a downstream notification failure.
- Before every send or write, recheck tenant status, conversation takeover, approval validity, connection status, and budget.
- Exit tests: worker crash after provider success, duplicate webhook, delayed callback, Redis restart, and expired OAuth all recover without silent loss.

13 / CHANNEL IMPLEMENTATION

WhatsApp integration

Provider path and onboarding

- Choose one approved implementation path for MVP. Suggested evaluation: Twilio as the initial provider adapter; compare onboarding effort and unit cost against direct Meta Cloud API before committing.
- For onboarding other businesses, follow the provider-supported ISV/Tech Provider route. A sandbox is for testing and does not replace production business onboarding. [S1]
- Each business owns or authorizes its sender. Store the provider account, business account, phone-number identifier, and organization binding; verify uniqueness.
- Show connection status, sender readiness, template status, and a send/receive test. Build a reconnect/support state for blocked setup.
- Keep a provider-neutral interface: verifyWebhook, normalizeInbound, sendText, sendTemplate, getMessageStatus, and disconnect.

Messaging policy implementation

- Track the latest inbound customer-message timestamp. The documented customer-service window is 24 hours; outside it, use an approved template rather than free-form text. Recheck current rules at launch. [S1]
- Record consent evidence and honor opt-outs. A booking inquiry is not blanket authorization for unrelated marketing. Enforce consent at send time. [S1]
- Store template identifier, language, purpose, approval status, and allowed variables. A paused or rejected template blocks the send and creates an operator task.
- Capture outbound accepted, delivered, read, and failed states where the provider exposes them. API acceptance is not proof the customer received the message.

Conversation handling

- MVP accepts text. If media or voice arrives, acknowledge the supported limitation and route to staff rather than pretending it was understood.
- Normalize phone numbers; keep customer identity scoped to the business. Verify signatures before tenant routing or processing.
- Before sending an AI reply, verify the conversation is still in automatic mode and the message has not been superseded.
- Recheck template, provider pricing, number migration/coexistence, and approval requirements before public launch; provider rules can change.

14 / CONNECTOR IMPLEMENTATION

Calendar, Sheets, email, and forms

Google authorization and Calendar

- Use server-side OAuth with state validation, short-lived authorization codes, and encrypted refresh-token storage. Request only scopes needed by the feature. Handle revoked/expired authorization explicitly. [S2, S3]
- Let the owner select a calendar and test read/write access. Store the calendar ID and timezone, not just its display name.
- Use free/busy results plus business hours, service duration, buffers, and minimum notice to generate slots. Create an event through the documented insert API. [S4]
- Store UTC instants and the IANA business timezone. Reconfirm ambiguous dates and account for daylight-saving changes.
- Lock and reserve the internal resource interval in Postgres, then recheck the external calendar. A local lock cannot stop external calendar edits: monitor conflicts and escalate them.
- Use a stable valid provider event ID when supported and reconcile after timeouts. Confirm only after successful creation; release local holds on expiry.

Sheets as a mirror, not the source of truth

- Owner selects a sheet and approves columns: internal lead ID, name, service, contact, status, booking time, and updated time.
- Use the internal lead ID to locate/update rows. Serialize writes per sheet and reconcile ambiguous inserts before retrying; a blind append can duplicate rows.
- Detect renamed tabs or changed headers and pause sync with a repair instruction. Keep Postgres records intact.
- Batch writes where practical, throttle by provider/account, and use backoff for quota errors. [S5]

Email and forms

- MVP email sends staff notifications from your verified transactional domain. Configure SPF/DKIM/DMARC with the provider and process bounce events.
- Sending as the business or reading its inbox requires a separate verified-domain or OAuth integration in phase 5; it is not included automatically.
- MVP forms use signed, expiring links and a fixed field schema. Validate server-side, rate-limit submissions, and prevent spreadsheet formula injection.
- Later third-party form submission requires an approved API or webhook adapter, explicit field mapping, and verified authorization. Do not promise arbitrary website form automation.

15 / BOUNDED INTELLIGENCE

AI behavior and evaluation

Use AI where interpretation helps

- Classify the request, extract customer details, summarize a conversation for staff, and draft an answer from approved business knowledge.
- Return schema-validated fields: intent, extracted_fields, missing_fields, evidence_message_ids, proposed_reply, and needs_human.
- Use ordinary code for price calculations, scheduling, authorization, consent, and action execution. AI cannot grant itself permissions.
- Treat a model confidence score as a weak signal. Missing evidence, invalid fields, or policy conflicts should trigger clarification or handoff.

Prompt and knowledge design

- Separate trusted system rules, owner-approved business settings, retrieved knowledge, and untrusted customer messages.
- Explicitly prevent customer text or uploaded documents from changing tool permissions, recipients, prices, or business policy.
- Start with a small approved FAQ set. Later add tenant-filtered document retrieval with source IDs, versioning, and access checks.
- If no approved answer exists, ask a targeted question or route to staff. Never invent opening hours, refunds, or availability.
- Send only the minimum necessary context to the model. Redact secrets and unnecessary personal data; bound conversation history and output length.

Evaluation and release gate

- Build at least 100 anonymized or synthetic cases: normal inquiries, incomplete details, typos, ambiguous dates, adversarial instructions, complaints, and provider failures.
- Measure field extraction accuracy, wrong-action rate, unnecessary handoff rate, unsupported factual claims, cost, and response latency.
- Proposed gate: at least 95% accuracy on required-field extraction in the representative test set; zero unauthorized actions in adversarial tests. These targets are not a guarantee of production accuracy.
- For Arabic launch, add native-speaker review of dialect, dates, mixed-language messages, and right-to-left display. Evaluate each supported language independently.
- Version prompts and model configuration. Run the regression set before changes and canary the change with a small tenant group.

16 / PRODUCTION REQUIREMENTS

Security, privacy, and tenant isolation

Access and secrets

- Enable row-level security on exposed tenant tables and test owner/operator permissions. Database service credentials can bypass protections, so keep them server-only and add worker-level tenant checks. [S7]
- Derive org_id from verified membership or trusted connector mapping. Every query, file lookup, export, and background action must enforce that scope.
- Use least-privilege OAuth scopes, encrypted credential storage, secret-manager references, key rotation, and restricted access to production credentials.
- Require MFA for platform administrators; offer it to business owners. Audit invitations, role changes, publishing, exports, and connector changes.
- Use signed webhooks, request size limits, CSRF protection where relevant, rate limits, secure cookies, and dependency/secret scanning.

Data lifecycle

- Proposed default: retain message content for 90 days and redacted run metadata for 180 days; let contracts and applicable obligations determine final values.
- Document data categories, subprocessors, regions, retention, deletion, backup expiry, and whether providers retain model inputs. Do not claim zero retention without verification.
- Provide tenant export and deletion workflows. Deletion revokes connections, stops scheduled work, removes active data, and schedules backup expiry under the documented policy.
- Separate platform analytics from customer message content. Logs contain IDs and error codes, not full conversations by default.
- Before a country launch, obtain a focused review of privacy, messaging, consumer terms, tax, and data-transfer obligations. This guide does not establish jurisdiction-specific legal compliance.

Security acceptance checks

- Attempt access to another tenant through every record ID, export, file URL, search endpoint, webhook binding, and job payload.
- Test prompt injection attempting data export, recipient changes, price overrides, and secret disclosure. Untrusted content must remain data.
- For later URL/webhook connectors, block private/internal IP ranges, validate redirects, allowlist destinations, and cap response sizes to prevent server-side request forgery.
- Maintain an incident checklist: stop affected actions, revoke credentials, preserve evidence, determine impact, notify under applicable obligations, and fix the root cause.

17 / REVENUE MODEL

Billing, pricing, and operating economics

Initial offer - hypotheses to test

- Paid pilot: proposed USD 99-199 per month plus an optional USD 100-300 setup fee. Define supported workflows and included usage before charging.
- After pilots, test one simple base plan with one sender and the three workflows. Add a higher plan only when customers need clear additional value.
- Show message/AI usage clearly. Pass through or price for provider costs using a transparent rule; do not offer unlimited automation before measuring costs.
- Custom connector work is a separate quote with delivery scope and ongoing support terms. Keep it out of the standard subscription promise.

Subscription implementation

- Select an eligible payment provider for the operating entity. Use hosted checkout and billing portal features where available.
- Store customer ID, subscription ID, status, billing period, plan, limits, and an internal entitlement version.
- Verify signed billing webhooks and deduplicate event IDs. Payment events can arrive out of order: reconcile authoritative subscription state before changing access. [S8]
- Count usage from immutable internal usage events with unique operation keys. Separate completed work from provider-billable attempts.
- On payment failure, notify the owner and follow a published grace policy. Keep data readable and explain paused automations; never silently discard customer requests.

Unit economics and spending controls

- Monthly contribution = subscription revenue minus messaging, model usage, email, variable hosting, payment fees, refunds, and directly attributable support cost.
- Illustration only: at USD 149 revenue and USD 40 variable cost, contribution is USD 109, or about 73%. Twenty customers produce USD 2,980 revenue and USD 2,180 contribution before fixed costs and taxes.
- Proposed early operating reserve: USD 150-500 per month for managed infrastructure and monitoring, excluding labor, legal review, setup, and variable usage. Replace this with vendor quotes before purchase.
- Track cost per completed workflow and support minutes per tenant. Alert at 80% of an agreed usage budget; apply the customer-approved overage or pause policy at the limit.
- Target positive contribution during pilots and at least 70% contribution margin before aggressive acquisition, including a realistic value for support time.

18 / RUN THE SERVICE

Environments, deployment, and operations

Environment setup

- Create separate development, staging, and production projects, databases, queues, storage buckets, credentials, webhook URLs, and billing modes.
- Use synthetic data in development. Production data cannot be copied into a developer laptop or staging without an approved, minimized process.
- Maintain an environment-variable manifest containing names and purposes only: database URL, auth settings, Redis URL, provider secret references, AI configuration, and webhook signing secrets.
- Provision resources through repeatable configuration or infrastructure-as-code once the pilot architecture is stable.

Continuous delivery

- On a pull request: type-check, lint, run domain tests, run tenant-isolation tests, validate workflow schemas, and scan for secrets.
- On staging deploy: apply migrations, start compatible workers, run provider sandbox checks, and execute end-to-end inquiry/booking/handoff tests.
- Use additive database changes first. Deploy compatible code, backfill data, then remove old fields in a later release.
- Release to one pilot tenant through a feature flag; monitor; expand only after the agreed observation window. Record deployment SHA and workflow/prompt versions.
- Rollback application code and switch new runs to a known workflow version. For database changes, use a tested forward repair when reversal risks data loss.

Operational visibility and recovery

- Monitor webhook acceptance, outbox age, queue age, action failures, uncertain actions, provider latency, token refresh errors, and cost per tenant.
- Proposed pilot alerts: oldest ready job above 2 minutes, failure rate above 5% over 15 minutes, or any cross-tenant access incident. Tune using real traffic.
- Use structured logs with request_id, org_id, run_id, and operation_key. Redact payload content and credential material.
- Enable database backups and point-in-time recovery if the plan supports it. Proposed production objectives: no more than 1 hour of data loss and 4 hours to recover; verify through restore drills.
- Document support hours, incident severity, owner contact, status updates, and escalation. A solo founder should not sell an unsupported 24/7 response guarantee.

19 / PHASE 1 - ITERATIONS 1 AND 2

Foundation build tasks

Iteration 1 - account and tenant foundation

- Create the repository, TypeScript packages, local setup instructions, CI pipeline, and environment configuration.
- Implement sign-up/sign-in, organizations, memberships, role checks, and private file access. Add initial database migrations and seed a synthetic business.
- Build dashboard navigation, empty states, loading states, and error presentation. Add audit recording for sensitive changes.
- Implement tenant-scoped service and business-config CRUD. Add server-side schema validation and cross-tenant tests.
- Done: two test businesses can use the same deployment without seeing each other's records; an operator cannot change billing or publish a workflow.

Iteration 2 - setup, connectors, and simulation

- Build the guided questionnaire with save/resume and a read-only workflow preview. Keep the draft configuration versioned.
- Implement connector adapter interfaces, encrypted credential references, OAuth callbacks, and destination selection.
- Build fake WhatsApp, calendar, sheet, and email adapters that record intended actions in test mode. Test mode must never send real messages.
- Connect one real test sender, one calendar, and one sheet in staging. Make disconnection and reauthorization visible in the dashboard.
- Done: an owner completes setup, sees exact proposed destinations, and runs a simulated inquiry end-to-end with no external writes.

Marketing and evidence during this phase

- Show the onboarding prototype to three committed businesses. Observe without coaching; record confusing questions and missing business rules.
- Publish one short workflow demo and collect demo requests. Use screenshots with fictional data and label unfinished features accurately.
- Start an objection log: price, account access, reliability, existing number, Arabic support, and staff control. Turn repeated objections into product or sales tasks.
- Phase gate: usable setup, tenant isolation passing, staging connectors working, and no unresolved production-provider blocker hidden from the pilot customers.

20 / PHASE 2 - ITERATIONS 3 TO 5

MVP workflow build tasks

Iteration 3 - lead capture

- Build verified webhook ingestion, event deduplication, transactional outbox, queue dispatch, and the W1 state machine.
- Implement structured AI extraction, clarification questions, lead storage, sheet mirroring, and staff email.
- Add run history with readable action results. Add consent/opt-out state and the message-window policy check.
- Done: a real staging message becomes one validated lead; replaying it does not duplicate business effects; a failed sheet write remains recoverable.

Iteration 4 - bookings and handoff

- Build W2 slot calculation, exact-time confirmation, internal interval reservation, calendar event creation, and reconciliation.
- Build W3 takeover, assignment, approval expiry, operator reply, and deliberate resume.
- Test two simultaneous booking requests, a changed external calendar, ambiguous dates, customer corrections, and a staff takeover during AI generation.
- Done: confirmations reference a real saved booking; failures explain what is pending; the bot stays silent after takeover.

Iteration 5 - release hardening and billing

- Implement action receipts, bounded retries, uncertain-result review, dead-letter handling, circuit breakers, and usage accounting.
- Implement hosted checkout integration, entitlements, billing webhook verification, grace periods, and cancellation.
- Finish the privacy/terms drafts, business onboarding checklist, export/delete paths, backup configuration, and operator runbooks.
- Run the 100-case AI set and end-to-end failure tests. Complete a backup restore in staging and a production credential review.
- Done: a paid pilot tenant can activate, operate, pause, recover, view usage, and cancel. Zero known critical security or duplicate-booking defects.

Sales tasks and phase gate

- Schedule onboarding for three pilot customers; confirm their baseline metrics, exact supported scope, and success criteria.
- Deliver a five-minute owner training video and a one-page staff guide covering takeover and failures.
- Gate: all three workflows pass acceptance checks, provider production access works, and support responsibility is assigned.

21 / PHASE 3 - ITERATIONS 6 AND 7

Paid pilots and product validation

Iteration 6 - supervised operation

- Start with one customer, one sender, and a small subset of real inquiries. Use staff approval for outward actions until the business confirms behavior.
- Review each early failure and a sample of successful runs daily. Label wrong extraction, wrong rule, provider failure, confusing wording, and missing workflow support.
- Fix repeated root causes before onboarding the next business. Keep a log of any manual founder work hidden behind the service.
- Train staff to pause, take over, and report an incorrect action. Rehearse an outage and a disconnected account.

Iteration 7 - renewal and usability

- Expand to three to five businesses only after the first customer has stable operation. Run at least two weeks with measured traffic per business.
- Compare the same workflow before and after: staff handling minutes, completed inquiries, confirmed bookings, handoffs, and missed follow-ups.
- Ask whether the customer will renew at the intended ongoing price. Record the reason for cancellation or non-renewal.
- Reduce onboarding questions or support steps that repeatedly confuse owners. Do not expand the template set during unresolved reliability issues.

Proposed pilot exit gate

- At least three paying businesses; at least two explicitly renew or commit to continued paid use after the pilot.
- At least 95% technical success on supported eligible runs across a meaningful sample; report denominator and handoffs separately. Target at least 100 eligible runs across the pilot group.
- Zero observed unauthorized actions or system-created duplicate bookings; all uncertain actions surfaced to staff.
- At least two businesses demonstrate a material improvement against their agreed baseline, such as 30% less handling time. This is a validation target, not a promise.
- Positive per-customer contribution and a manageable founder support load. If it fails, improve the workflow or narrow the customer segment.

Marketing output

- With permission, produce one case study with baseline, sample size, dates, outcome, and limitations. Avoid extrapolating a small pilot to all businesses.
- Ask satisfied customers for a referral. Use their exact approved testimonial rather than AI-generated praise.

22 / PHASE 4 - ITERATIONS 8 AND 9

Self-service product and second template

Iteration 8 - reduce setup effort

- Replace manual provider onboarding steps only where the official provider flow supports it. Display verification progress and clear repair instructions.
- Add a setup checklist, sample data, contextual help, template recommendations, and one-click test scenarios.
- Add owner-visible workflow change history, run search, configurable notification recipients, and usage warnings.
- Instrument where setup stalls: account creation, provider connection, destination mapping, test failure, or payment.
- Done: most qualified pilot-like customers can reach a successful test with limited support; target 80% and measure actual founder minutes.

Iteration 9 - carefully expand configuration

- Add editable required fields, service-specific questions, business-hour branches, and approval rules through forms.
- Choose the next template from paid requests. Prefer a closely related process such as quotation review or recurring cleaning inquiries.
- Add cancellation/rescheduling only with explicit customer identity checks, provider update receipts, and stale-action tests.
- If demand supports it, add validated Arabic support and localized setup copy with native-speaker QA.
- Done: the new template reuses the same engine and connectors, with its own acceptance suite and no customer-specific code fork.

Marketing and exit gate

- Publish industry-specific examples and practical guides answering real setup objections. Create a live demo business with synthetic data.
- Test referrals and one partnership channel, such as agencies serving local businesses. Document onboarding responsibility and support boundaries.
- Proposed gate: ten paying businesses, at least two billing cycles of retention evidence, stable contribution margin, and declining support time per account.
- Delay paid advertising if activation or renewal is weak. More leads will otherwise amplify a product problem.

23 / PHASE 5 - ITERATIONS 10 TO 12

Configurable workflow platform

Iteration 10 - bounded workflow editor

- Add a step-list editor using approved actions: form trigger, message trigger, condition, field collection, save record, notify, wait, and approval.
- Expose field mappings with example values and type checks. Let owners reorder only compatible actions and show required upstream data.
- Validate the graph, simulate branches, preview external effects, and publish an immutable version.
- Done: a customer builds a supported lead workflow without code; invalid mappings and missing permissions block publication.

Iteration 11 - new connectors and triggers

- Add the most requested CRM and inbound business email integration after confirming OAuth/security requirements and support cost.
- Add hosted form templates and signed inbound webhooks. Add generic outbound webhooks only with destination restrictions and permission review.
- Use a connector registry with action schemas, scopes, auth type, version, retry rules, rate limits, and test fixtures.
- Done: connector failures are classified consistently; revoked access pauses affected actions; customer data remains isolated.

Iteration 12 - AI-generated drafts

- Turn questionnaire answers and a plain-language goal into a proposed graph made only of registered actions.
- The planner may select templates and suggest mappings, but cannot create arbitrary code, credentials, or new tool permissions.
- Show assumptions, unsupported steps, destination accounts, estimated usage, and approval points. Ask targeted clarification questions.
- Run schema checks, permission checks, static graph validation, and sandbox examples before allowing owner publication.
- Done: evaluate at least 50 representative setup requests. Reject unsupported requests clearly and require zero unauthorized action types in generated plans.

Market gate

- Sell customizable workflows within named supported categories. Publish an honest integration catalog and supported-action list.
- Advance only when repeated paid requests justify platform flexibility and template-based workflows continue to perform reliably.

24 / PHASES 6 AND 7

Broader vision and long-term expansion

Phase 6 - iterations 13 and 14

- Add industry packs for validated workflows: agency onboarding, repair-service intake, property inquiries, or document requests. Research each industry before release.
- Each pack includes questionnaire fields, service rules, knowledge starter content, workflow templates, demo data, tests, and an onboarding guide.
- Add multiple locations, staff resources, assignment rules, location-specific hours, and a central manager view. Model capacity before supporting multi-crew bookings.
- Add partner accounts with explicit client membership, per-client billing/usage, template deployment, and audited access. Do not let partners access unrelated organizations.
- Pilot a curated marketplace with reviewed templates, declared permissions, version compatibility, support ownership, and a way to disable unsafe versions.

Phase 7 - iteration 15 onward

- Expand into approved end-to-end processes: inquiry to quote to approval to job record to follow-up; onboarding to document collection to staff task creation.
- Add document extraction with malware scanning, per-field evidence, and human review for consequential fields.
- Consider voice, multi-channel history, enterprise SSO, custom roles, regional deployment, data-retention controls, and contractual service levels when buyers fund the complexity.
- Support advanced workflow scheduling, bounded batch processing, reusable subflows, and compensation steps for reversible actions.
- Introduce process improvement suggestions based on measured outcomes. Suggestions remain drafts; never silently change a live business procedure.

Expansion gates and organization

- Require several paying customers requesting the same capability, a measurable outcome, a support plan, and acceptable unit economics before each expansion.
- Hire or contract support when operational coverage limits growth; add engineering capacity when connector maintenance prevents reliable delivery.
- For enterprise commitments, fund security review, incident coverage, disaster recovery, and contract review before promising them.
- Keep the long-term promise precise: automate supported, authorized digital steps and surface exceptions. Full business automation remains bounded by data quality, access, physical work, and human judgment.

25 / QUALITY GATES

Test plan and release acceptance

Domain and integration tests

- Test price math in currency minor units, required fields, service-area rules, holidays, slot duration, buffers, timezone conversion, and approval expiry.
- Test OAuth callback mismatch, expired tokens, revoked access, malformed payloads, failed signature checks, duplicate events, and provider quota responses.
- Use adapter contract tests against recorded/synthetic fixtures. Run a small live sandbox smoke test before release; never load-test a provider without permission.
- End-to-end tests cover setup -> test -> publish -> inquiry -> booking -> confirmation, plus handoff, cancellation, and billing suspension.

Failure and security tests

- Crash after external write but before receipt save; simulate response timeout; replay callbacks out of order; restart worker/Redis; verify reconciliation.
- Simultaneous slot requests, overlapping appointments, expired holds, and external calendar edits must not create false booking promises.
- Test every tenant boundary, role restriction, signed form link, export, file download, and background job authorization.
- Test prompt injection, unexpected language, excessive input, spreadsheet formula injection, and unsupported media.
- Test takeover races: queued model output, pending send, and resumed conversations. Confirm the final authority check happens immediately before the send.

Performance and usability

- Proposed staging load: 20 concurrent conversations and a burst of 100 duplicate/reordered events. Measure queue delay and resource usage before setting customer limits.
- Target webhook acknowledgment below 2 seconds after durable acceptance and ordinary response under 15 seconds at the tested load. Treat these as internal goals, not promised SLAs.
- Observe five nontechnical users completing setup and resolving a failure. Review keyboard/mobile behavior and localized layouts.
- Release checklist: migrations compatible, backups restorable, alerts working, critical defects closed, AI evaluation passed, known limitations documented, and rollback rehearsed.
- Keep evidence: build SHA, test report, dataset version, reviewer, sample size, unresolved issues, and approval date.

26 / ACQUISITION THROUGHOUT DELIVERY

Launch and marketing playbook

Positioning and audience

- Initial message: handle cleaning inquiries, capture customer details, and coordinate bookings through WhatsApp using your business rules.
- Use a specific before/after demonstration. Show the owner exactly what staff can control and what happens when automation needs help.
- Sell to the owner or operations lead. Qualify for sufficient recurring inquiries, repeatable services, willingness to connect accounts, and an internal champion.

Before and during pilots

- Phase 0: interview-led outreach, referrals, one landing page, and a demo appointment form.
- Phases 1-2: share progress demos with committed prospects, collect objections, and reserve pilot onboarding slots.
- Phase 3: create a permission-based case study and referral request; document support effort and outcomes honestly.
- Use direct, personalized outreach where appropriate; respect channel rules and opt-outs. Do not use the customer automation product for unsolicited bulk WhatsApp campaigns.

After retention evidence

- Phase 4: publish comparison pages and setup guides for the chosen industry; offer webinars or live demonstrations with a relevant partner.
- Launch on Product Hunt when setup works and there is a clear demo, onboarding help, and real customer evidence. Treat it as awareness, not proof of product-market fit.
- Phase 5: publish connector recipes and customer workflow examples. Introduce an agency partner offer only after documenting ownership and support terms.
- Run a small paid-acquisition test only after measuring activation and retention. Set a capped budget and a stop date before launch.

Weekly founder sales routine

- Contact 10-15 qualified prospects, hold 3-5 discovery/demo calls where possible, review objections, and follow up on agreed next steps.
- Track source -> qualified lead -> demo attended -> paid pilot -> activated -> renewed. Record conversion denominators and acquisition cost.
- Avoid feature-led content with no buyer outcome. Each marketing asset should answer a real objection or show a completed workflow.
- Do not invent testimonials, savings claims, logos, or customer numbers. Obtain approval for anything identifying a customer.

27 / MEASURE BUSINESS VALUE

Metrics, experiments, and decision rules

Product metrics and definitions

- Activation: a paid tenant connects required accounts, publishes a tested workflow, and completes its first real supported run.
- Technical success rate: eligible runs completing their intended technical steps divided by all eligible supported runs. Report provider failures and human handoffs separately.
- Business completion: qualified inquiries reaching the agreed business outcome, such as a saved lead or confirmed appointment.
- Time saved: compare measured handling time before and after for comparable requests. Report the sample and manual work still required.
- Retention: paying organizations remaining after each billing period. Report both customer count and recurring revenue retention.
- Support burden: founder/operator minutes per active tenant per week. Include manual repairs and onboarding labor.

Analytics events

- Track setup_started, connection_succeeded, connection_failed, workflow_test_passed, workflow_published, first_run_completed, handoff_requested, action_uncertain, and subscription_cancelled.
- Record event ID, tenant ID, timestamp, plan, template version, and safe properties. Exclude message bodies and unnecessary customer identifiers.
- Separate product analytics from authoritative billing records. Deduplicate analytics and document metric queries.

Experiments and stop rules

- Test questionnaire wording against setup completion and support time. Do not vary business safety rules casually.
- Test pricing with new prospect cohorts and clearly disclosed terms; compare contribution and renewal, not just initial signups.
- If fewer than two of the first three paid pilots want to renew, investigate value and fit before adding channels.
- If most requests require bespoke code, narrow the niche or introduce a service business model with explicit pricing.
- If reliability falls below the pilot target, freeze expansion and resolve the largest failure category.
- For paid ads, compare fully loaded acquisition cost with monthly contribution. Proposed goal: payback within three months; pause a channel that cannot plausibly reach it.

28 / OPERATING PLAYBOOKS

Support, customer success, and risk register

Common support procedures

- Disconnected Google account: stop dependent actions, notify owner, reconnect, verify the selected calendar/sheet, then resume eligible queued work.
- Booking result uncertain: inspect action receipt and provider event before retrying; contact staff when reconciliation cannot prove the outcome.
- Wrong answer: pause affected conversation, correct the business knowledge/rule, communicate through staff, add the case to regression tests.
- Unexpected cost spike: cap the affected workflow, inspect loops/retries and tenant usage, preserve evidence, then adjust limits or pricing transparently.
- Cancellation: confirm the effective date, stop future runs at the agreed time, explain pending bookings, provide export, and follow the retention policy.

Risk register with response

- Provider approval delay - founder owns access checks in phase 0; maintain a clearly labeled simulator and do not charge for unavailable activation.
- Low willingness to pay - founder owns paid pilots; narrow the customer segment or stop before broad development.
- Duplicate or incorrect actions - developer owns receipts, state checks, and fault tests; ambiguous results go to review.
- Cross-tenant exposure - developer plus independent reviewer own isolation tests; any confirmed issue blocks release.
- Too much custom work - founder tracks per-account engineering hours; quote separately or reject unsupported scope.
- Unprofitable usage - founder tracks contribution, budgets, and support time; change limits or packaging before scaling.
- Single-founder outage/support gap - publish realistic support hours; maintain a backup operator or contractor before contractual coverage expands.

Success routine

- During pilots: daily run review and weekly owner call. After stabilization: monthly outcome report and integration-health check.
- Report completed work, exceptions, usage, and measured value. Ask which repeated manual task remains painful.
- Maintain an issue log with severity, affected tenants, workaround, owner, due date, and regression-test link.
- Review provider/API changes monthly and before major releases. Assign one owner to every connector and public claim.

29 / ACTIONABLE STARTING PLAN

First 30 days and delivery checklist

Week 1

- Confirm the initial business category and language. Write the one-sentence offer and three supported workflows.
- Contact the first 15 prospects and complete at least five interviews. Map their process and collect authorized anonymized examples.
- Start WhatsApp provider eligibility, Google OAuth, and payment-provider checks. Record external requirements and expected delays.

Week 2

- Finish ten interviews, demonstrate the proposed workflow, and seek three paid-pilot commitments.
- Finalize the MVP boundary, baseline metrics, draft pricing, and pilot terms. Pause broad development if the validation gate fails.
- Create the repository, environment manifest, architecture decision log, initial schema, and CI pipeline after the offer is credible.

Week 3

- Implement organizations, roles, tenant checks, business settings, and the basic dashboard.
- Add synthetic demo data and fake connector adapters. Build the first end-to-end simulated lead workflow.
- Review the questionnaire with pilot customers; remove questions that do not affect supported behavior.

Week 4

- Build save/resume onboarding, provider authorization callbacks, connection status, and workflow preview.
- Complete the first staging sender/calendar/sheet test. Continue approval work and communicate any external blocker.
- Demo progress to committed customers and agree the pilot onboarding sequence. Do not imply the complete MVP is finished by day 30.

Definition of done for every tracked task

- The intended user outcome and scope are written in plain language.
- Required data, permissions, error behavior, and usage impact are documented.
- Implementation is reviewed; relevant acceptance/failure tests pass; UI states are understandable.
- Documentation and support instructions are updated; rollout and rollback are clear.
- Completion evidence is attached. Mark blocked work honestly rather than treating a successful demo as production readiness.

30 / SHARED LANGUAGE

Glossary and decision log

Plain-language glossary

- Tenant: one business using the product. Tenant isolation: keeping its data and actions separate from every other business.
- API: a supported way for one application to ask another application to do something. Connector: your code that handles one provider's API.
- OAuth: a business authorizes access to an account without giving your product its password. Scope: the specific permission granted.
- Webhook: a provider sends your server a message when something happens. Queue: work waiting for a background worker.
- Idempotency: repeating the same intended operation does not create an extra business effect. Reconciliation: checking what actually happened after an uncertain result.
- Workflow: a saved sequence of rules and actions. Run: one execution of that workflow. Template: a reusable starting workflow.
- Outbox: database records of work that still needs dispatch. Action receipt: a record linking an intended action to its provider result.
- RLS: database rules controlling which rows a user may access. RBAC: permissions based on roles.
- MVP: the smallest useful paid product. Exit gate: evidence required before the next phase. Contribution: revenue remaining after variable delivery costs.

Decisions to confirm before phase 1

- D01 - niche and country: default cleaning businesses; decide from interviews. Owner: founder.
- D02 - WhatsApp provider path: evaluate supported third-party onboarding and full cost. Owner: developer/founder.
- D03 - first language and timezone: choose from pilots; Arabic requires its own QA. Owner: founder.
- D04 - payment provider and entity: verify eligibility and billing obligations. Owner: founder with appropriate professional advice.
- D05 - hosting/data regions and retention: decide from customer obligations, provider capabilities, and budget. Owner: developer/founder.
- D06 - support hours, pilot price, caps, and refund terms: agree before accepting payment. Owner: founder.
- D07 - calendar ownership and conflict policy: confirm who else edits the calendar and how conflicts reach staff. Owner: pilot business and developer.
- For each decision, record date, options, choice, reason, evidence, owner, and the condition that would justify changing it.

31 / SOURCES

Official references and implementation notes

Integration references - checked 14 September 2026

- [S1] Twilio: WhatsApp Business Platform overview. Business onboarding, message windows, consent, templates, and callbacks. https://www.twilio.com/docs/whatsapp/api
- [S2] Google: OAuth 2.0 for web server applications. Authorization and token handling. https://developers.google.com/identity/protocols/oauth2/web-server
- [S3] Google: Calendar API authorization scopes. Select permissions according to implemented features. https://developers.google.com/workspace/calendar/api/auth
- [S4] Google: Calendar events.insert reference. Event creation contract. https://developers.google.com/workspace/calendar/api/v3/reference/events/insert
- [S5] Google: Sheets API usage limits. Quotas and backoff guidance. https://developers.google.com/workspace/sheets/api/limits
- [S6] BullMQ: idempotent jobs and auto-removal. Retry design and limits of job-ID deduplication. https://docs.bullmq.io/patterns/idempotent-jobs | https://docs.bullmq.io/guide/queues/auto-removal-of-jobs
- [S7] Supabase: Row Level Security. Database access policies and service credential behavior. https://supabase.com/docs/guides/database/postgres/row-level-security
- [S8] Stripe: webhooks. Signature verification, duplicate events, and event ordering. Reference only; provider choice remains subject to eligibility. https://docs.stripe.com/webhooks

How sources are used

- Source markers identify provider behavior used in this plan. Architecture, schemas, schedules, acceptance targets, pricing, and marketing experiments are proposed design decisions.
- Pin the actual SDK/API versions during implementation and record them in the repository. Do not infer that an unversioned documentation URL freezes behavior.
- The direct Meta embedded-signup page could not be retrieved during this research. Confirm current Meta and provider onboarding requirements through the selected official provider path before public rollout.
- Recheck messaging rules, consent requirements, OAuth verification, payment eligibility, API quotas, and vendor prices before launch. No competitor revenue or profitability is asserted.